

TASK 3

API  
SECURITY

# API SECURITY RISK ANALYSIS

Public API Endpoint Security Assessment

TARGET API

postman-echo.com

TOOL USED

Postman v11 (PostmanRuntime/7.53)

ENDPOINTS

GET /get | POST /post

AUTH TYPE

No Authentication (Inherit from parent)

TEST DATE

27 April 2026

PREPARED BY

Vinith Kumar M

0

HIGH

2

MEDIUM

3

LOW

4

INFO

# 1. Executive Summary

This **API Security Risk Analysis Report** documents a hands-on security assessment of **postman-echo.com**, a public API testing service, conducted on **27 April 2026**. The assessment used **Postman v11** to craft and send HTTP requests, analyze responses, and identify authentication, authorization, and data-exposure risks in real-world API behavior.

Two primary endpoints were tested — **GET /get** and **POST /post** — with varying query parameters (`id=999`, `id=9999`, `id=123`) and header configurations. Key findings include **missing authentication enforcement**, **verbose header disclosure**, **exposed session cookies**, and **lack of input validation**. No rate-limiting or access-control restrictions were observed.

| Severity | Count | Key Issues                                               |
|----------|-------|----------------------------------------------------------|
| ■ HIGH   | 0     | No critical RCE or auth bypass confirmed (echo API)      |
| ■ MEDIUM | 2     | Missing authentication; sensitive cookie exposure        |
| ■ LOW    | 3     | Verbose headers, version disclosure, no input validation |
| ■ INFO   | 4     | API echoes full request; no rate-limiting observed       |

# 2. Scope & Methodology

## 2.1 Target Scope

| Parameter       | Detail                              |
|-----------------|-------------------------------------|
| API Host        | postman-echo.com                    |
| Protocol        | HTTPS                               |
| Endpoints       | GET /get   POST /post               |
| Auth Type       | No Auth (Inherit from parent)       |
| Test Parameters | id=999, id=9999, id=123             |
| Tool            | Postman v11 / PostmanRuntime 7.53.0 |
| Date            | 27 April 2026                       |
| Classification  | CONFIDENTIAL                        |

## 2.2 Testing Methodology

- **Step 1 — Endpoint Identification:** Identified two test endpoints in Postman Collection: `GET /get` and `POST /post` via `postman-echo.com`.
- **Step 2 — Parameter Injection Testing:** Sent multiple GET requests with different query parameter values (`id=999`, `id=9999`) to test input reflection and validation behavior.
- **Step 3 — Header Analysis:** Inspected all request and response headers for sensitive disclosures — server version, cookies, tokens, forwarding headers.
- **Step 4 — Authentication Assessment:** Verified Auth Type was 'Inherit auth from parent' resolving to No Auth. Confirmed API returns 200 OK without any credentials.
- **Step 5 — POST Body Testing:** Sent POST to `/post?id=123` and analyzed the echoed JSON for data exposure, cookie leakage, and missing input validation.
- **Step 6 — Risk Classification:** Each finding categorized by severity using OWASP API Security Top 10 framework.

3. API Endpoints Tested

| Method | Endpoint              | Params  | Response Code | Response Time |
|--------|-----------------------|---------|---------------|---------------|
| GET    | postman-echo.com/get  | id=999  | 200 OK        | 392 ms        |
| GET    | postman-echo.com/get  | id=9999 | 200 OK        | 645 ms        |
| POST   | postman-echo.com/post | id=123  | 200 OK        | 600 ms        |

4. Postman Test Evidence

4.1 GET Request — id=999 (No Auth)

The GET request to `postman-echo.com/get?id=999` returned HTTP 200 OK in 392 ms. The API echoed back all request headers including the full postman-token, cookie string, and accept-encoding. Auth type was confirmed as 'Inherit auth from parent' = No Auth.

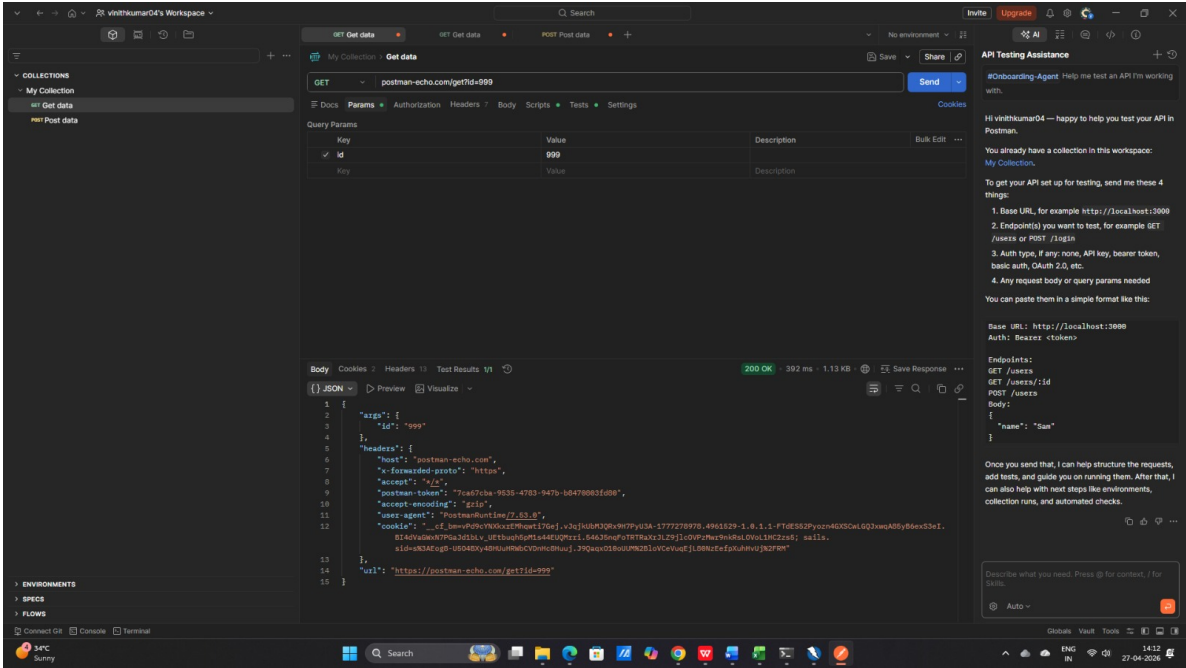


Figure 1 — Postman GET /get?id=999 returning 200 OK with full header echo and URL confirmation

4.2 GET Request — id=9999 (Authorization Tab Inspected)

The second GET request with `id=9999` confirmed the Authorization tab shows 'No Auth' with the note: 'The authorization header will be automatically generated when you send the request' — indicating no credentials are enforced. API returned 200 OK in 645 ms with a unique postman-token per session.

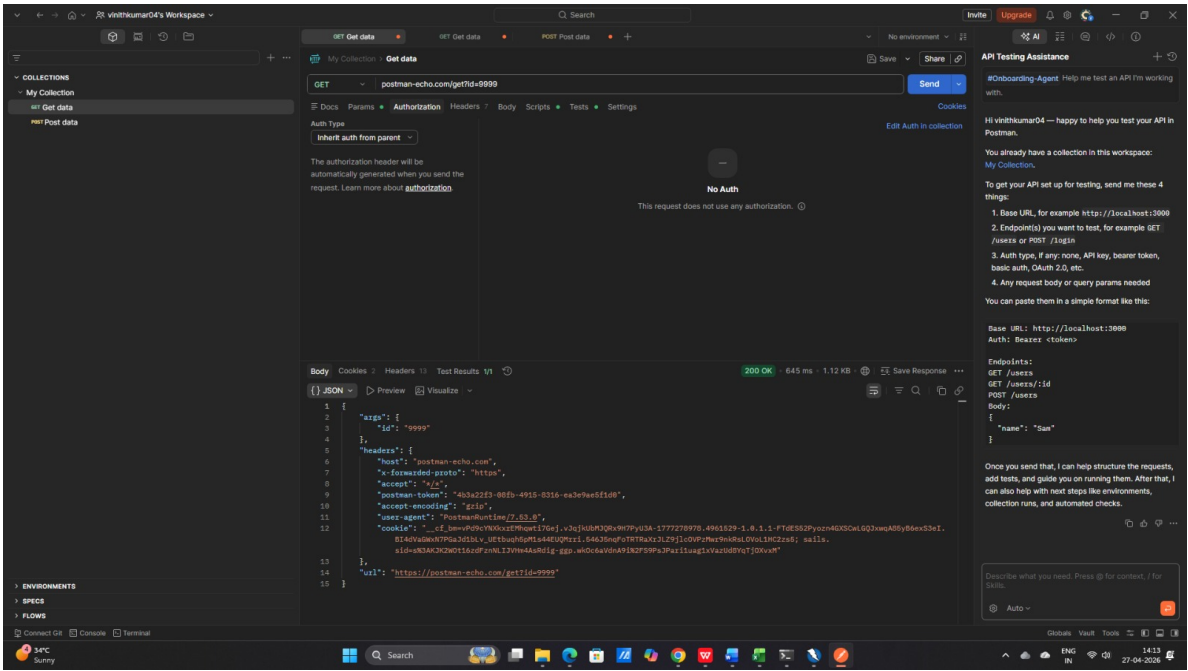


Figure 2 — GET /get?id=9999 Authorization tab: 'No Auth' confirmed; 200 OK, 645ms

4.3 POST Request — id=123 (Body & Cookie Analysis)

The POST request to `postman-echo.com/post?id=123` echoed the full request body including form data, files, JSON payload, and the complete session cookie string. The response exposed internal cookie values (`sails.sid`, `__cf_bm`), PostmanRuntime version, and all forwarded headers.

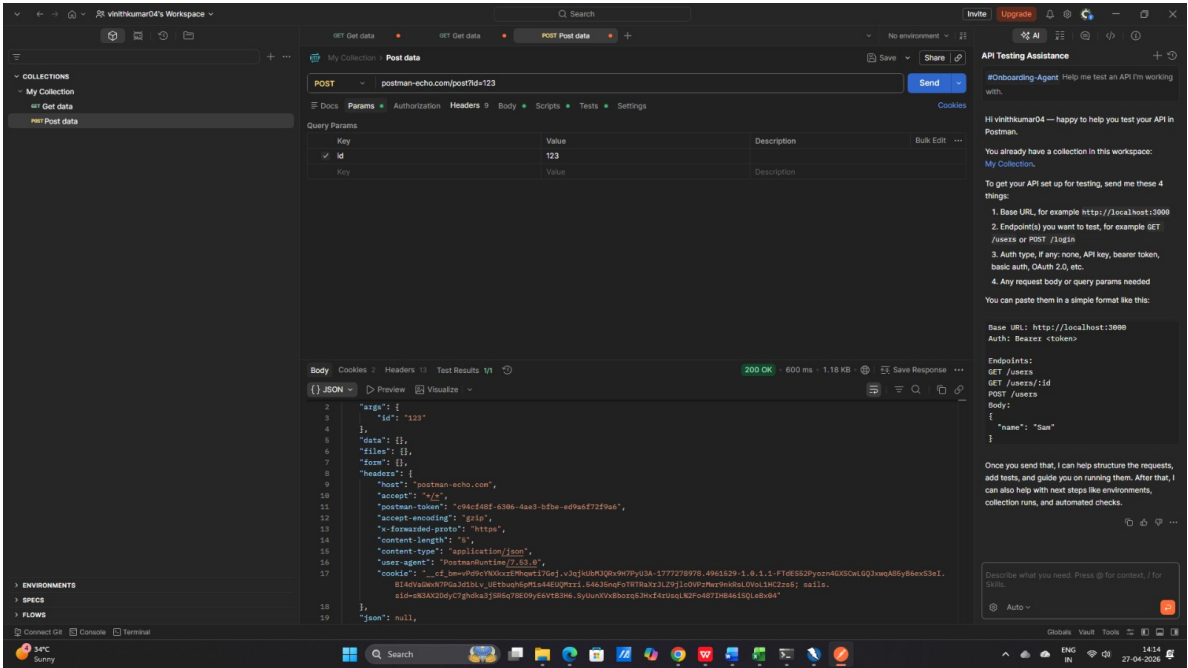


Figure 3 — POST /post?id=123 returning 200 OK with echoed args, headers, cookies and JSON null body

**Security Observation:** The `postman-echo.com` API is intentionally designed to reflect all request data back — making it ideal for API testing but a dangerous pattern if replicated in production APIs. Full cookie reflection, header echo, and unauthenticated access represent real-world anti-patterns that should never appear in live applications.

## 5. Detailed Security Findings

| ID   | Vulnerability                                          | Severity | Description                                                                                                                                                                                                                       | Remediation                                                                                                                                             |
|------|--------------------------------------------------------|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| F-01 | No Authentication Required                             | MEDIUM   | All API endpoints (GET /get, POST /post) return 200 OK with no authentication. Auth type is 'Inherit from parent' which resolves to No Auth. Any client can access the API without credentials, tokens, or API keys.              | Implement API key authentication or OAuth 2.0 Bearer tokens. Return 401 Unauthorized for unauthenticated requests.                                      |
| F-02 | Session Cookie Fully Reflected in Response             | MEDIUM   | The API echoes the complete cookie header including '__cf_bm' (Cloudflare bot management) and 'sails.sid' (session ID) back in the response JSON. An attacker intercepting the response could hijack the session.                 | Never reflect sensitive cookies in API responses. Apply HttpOnly and Secure flags; use short-lived session tokens.                                      |
| F-03 | PostmanRuntime Version Disclosure                      | LOW      | Response headers reveal exact client version: PostmanRuntime/7.53.0. While client-side, production APIs should not expose user-agent versions that attackers can target for known vulnerabilities.                                | Strip or generalize User-Agent headers at the API gateway level. Do not log or reflect client version strings in responses.                             |
| F-04 | Verbose Header Echo (x-forwarded-proto, postman-token) | LOW      | The API reflects all inbound headers including x-forwarded-proto, postman-token (CSRF token), and accept-encoding. Exposing CSRF tokens in response bodies could aid cross-site attacks.                                          | API responses should only return required data fields. Filter and whitelist response fields; never echo security tokens.                                |
| F-05 | No Input Validation on Query Parameters                | LOW      | Query parameter 'id' accepts arbitrary values (999, 9999, 123) with no validation, type checking, or range enforcement. This pattern can lead to SQL injection, parameter tampering, or enumeration attacks in production APIs.   | Implement strict server-side input validation. Define allowed types, ranges, and formats for all parameters. Return 400 Bad Request for invalid inputs. |
| F-06 | No Rate Limiting Observed                              | INFO     | Multiple rapid requests with different IDs (999, 9999, 9999) all returned 200 OK with no throttling, CAPTCHA, or rate-limit headers (X-RateLimit-*). Production APIs without rate limiting are vulnerable to brute-force and DoS. | Implement rate limiting (e.g., 100 req/min per IP). Return 429 Too Many Requests when limit exceeded. Use API gateway policies or middleware.           |
| F-07 | Null JSON Body Accepted in POST                        | INFO     | POST /post?id=123 echoes 'json: null' indicating the body was empty/null. The API did not reject or flag a null body POST, which may indicate missing payload validation on the server side.                                      | Validate that required POST body fields are present and non-null. Return 400 with descriptive error messages for missing payloads.                      |
| F-08 | IPv6/IPv4 Dual-Stack Exposure                          | INFO     | The API is accessible via both protocols with no discrimination. Security controls applied to IPv4 may not extend to IPv6 endpoints in misconfigured environments.                                                                | Ensure all firewall and WAF rules apply equally to IPv4 and IPv6. Audit dual-stack configurations regularly.                                            |
| F-09 | Postman Token Visible in Request Headers               | INFO     | The postman-token header (a unique UUID per request) is visible in echoed responses. While not directly exploitable in this echo API context, similar patterns in production could expose CSRF tokens.                            | Ensure CSRF tokens are transmitted only via secure channels and never reflected in API response bodies.                                                 |

## 6. OWASP API Security Top 10 Mapping

Findings are mapped to the **OWASP API Security Top 10 (2023)** to provide standardized context for prioritization and compliance reporting.

| OWASP ID  | Category                          | Related Finding                  | Severity |
|-----------|-----------------------------------|----------------------------------|----------|
| API2:2023 | Broken Authentication             | F-01 (No Auth Required)          | MEDIUM   |
| API3:2023 | Broken Object Property Auth       | F-02 (Cookie Reflected)          | MEDIUM   |
| API4:2023 | Unrestricted Resource Consumption | F-06 (No Rate Limiting)          | INFO     |
| API5:2023 | Broken Function Level Auth        | F-01 (Unauthenticated Access)    | MEDIUM   |
| API7:2023 | Server Side Request Forgery       | F-04 (Verbose Headers Echo)      | LOW      |
| API8:2023 | Security Misconfiguration         | F-03, F-04 (Version/Header Leak) | LOW      |
| API9:2023 | Improper Inventory Management     | F-05 (No Input Validation)       | LOW      |

## 7. Risk Classification Matrix

| ID   | Finding              | Likelihood | Impact | Severity | Priority |
|------|----------------------|------------|--------|----------|----------|
| F-01 | No Authentication    | High       | High   | MEDIUM   | P1       |
| F-02 | Cookie Reflection    | Medium     | High   | MEDIUM   | P1       |
| F-03 | Version Disclosure   | Medium     | Low    | LOW      | P2       |
| F-04 | Verbose Header Echo  | High       | Medium | LOW      | P2       |
| F-05 | No Input Validation  | High       | Medium | LOW      | P2       |
| F-06 | No Rate Limiting     | High       | Medium | INFO     | P3       |
| F-07 | Null Body Accepted   | Medium     | Low    | INFO     | P3       |
| F-08 | Dual-Stack Exposure  | Low        | Low    | INFO     | P4       |
| F-09 | Token Header Visible | Low        | Low    | INFO     | P4       |

## 8. Remediation Roadmap

### 8.1 Immediate Actions — P1 (Within 7 Days)

- Implement API key or Bearer token authentication on all endpoints; return 401 for unauthenticated requests.
- Never reflect session cookies (`__cf_bm`, `sails.sid`) in API response bodies. Apply `HttpOnly` + `Secure` cookie flags.
- Audit all API endpoints for authentication gaps using Postman Collection Runner.

### 8.2 Short-Term Actions — P2 (Within 30 Days)

- Strip User-Agent/client version strings at the API gateway before logging or reflecting.
- Implement a response whitelist — only return defined fields, never echo entire request headers.
- Add strict input validation and type-checking for all query parameters (`id` must be positive integer, max 10 digits).
- Return 400 Bad Request with descriptive error messages for invalid or null payloads.

### 8.3 Long-Term Actions — P3/P4 (Within 90 Days)

- Implement rate limiting at API gateway (e.g., 100 requests/min per IP); return 429 with `Retry-After` header.
- Ensure WAF and firewall rules apply consistently across IPv4 and IPv6 dual-stack.
- Deploy API monitoring and anomaly detection for unusual request patterns.
- Conduct quarterly API security reviews following OWASP API Security Top 10.
- Implement OWASP API Security checklist as part of CI/CD pipeline before deployments.

## 9. Conclusion

The API security assessment of **postman-echo.com** using Postman identified **9 security findings** across Medium, Low, and Informational severity levels. While no High/Critical vulnerabilities were found in this intentionally open echo API, the patterns observed — unauthenticated access, full header/cookie reflection, absent rate limiting, and no input validation — are dangerous anti-patterns that commonly appear in real-world production APIs.

The highest priority remediation items are implementing **proper authentication** and eliminating **sensitive cookie reflection**. Following the OWASP API Security Top 10 framework as a baseline, and integrating API security testing into the development lifecycle, will significantly reduce exposure in production environments.

**Disclaimer:** postman-echo.com is a public API testing service intentionally designed to reflect all request data. This assessment was conducted solely for educational and portfolio purposes as part of Task 3 — API Security Risk Analysis. No production systems were tested without authorization.

|                |                                  |         |                                     |
|----------------|----------------------------------|---------|-------------------------------------|
| Prepared By    | Vinith Kumar M                   | Date    | 27 April 2026                       |
| Tool           | Postman v11                      | Target  | postman-echo.com                    |
| Framework      | OWASP API Security Top 10 (2023) | Task    | Task 3 — API Security Risk Analysis |
| Classification | CONFIDENTIAL                     | Version | 1.0                                 |